SynthOmnicon Framework Guide

March 20, 2026

0.1 Overview

SynthOmnicon is a Python implementation of the **Unified Synthonicon** framework described in `QUANTSYNTHONICON.md`. It provides computational tools for analyzing self-organizing chemical systems using **ten formal primitives** that span molecular, supramolecular, and temporal domains.

0.2 The Eleven Primitives (v0.4.0)

The framework is built on eleven core primitives. The first ten cover all classical and quantum systems; the eleventh (Ω) is a quantum extension for topologically protected states — it is optional and defaults to trivial for all classical synthons.

Full Unified Notation: ; T; R; P; F; K; G; Γ; Φ; S; $\Omega\rangle(\Omega omitted for classical systems)$

0.2.1 Key Extensions in v2.0

1. **Kinetic Character (K)** — Separates thermodynamic and kinetic fidelity
2. **Extended Interaction Grammar (Γ)** — Boolean algebra for partner selection (AND/OR/SEQUENTIAL)
3. **Criticality Phase (Φ)** — Identifies scale-free systems at G-D degeneracy
4. **Refined Polarity (P)** — Distinguishes symmetric vs. pseudosymmetric self-complementarity

0.2.2 Composition Axioms (v2.1)

The framework validates against **seven axioms** — five composition axioms from QUANTSYNTHONICON.md Section IV, plus two new grounding axioms:

Composition Axioms (1-5):

1. **Axiom 1 (Cyclic Closure):** T_ + P_± → F ≥ F_eth
2. **Axiom 2 (Local Grammar Barrier):** G_ + Γ_⊗→ no global propagation
3. **Axiom 3 (Cooperative Induction):** Superlinear induction → G_ reclassification
4. **Axiom 4 (Sequential Grammar):** Γ_→ requires D_∞ or R_
5. **Axiom 5 (Criticality):** At criticality, G and D degenerate

Grounding Axioms (6-7) — NEW in v2.1: 6. **Axiom 6 (Temporal Grounding):** D_∞ requires closed cycle with reset mechanism 7. **Axiom 7 (Cyclic Topology Grounding):** T_ requires named closing bond/interaction

Primitive	Symbol	Description	Values
Dimensionality	D	Coordinate set of operation	D_∧ (molecular), D_ (supramolecular), D_∞ (temporal), hybrid
Topology	T	Internal connectivity pattern	T_ (cyclic), T_ (chain), T_ (hub/node), T_ (cage), $T_{\cup}$ (bowl), T_
Recognition Mode	R	Physical interaction mechanism	$R_{\subseteq}$ (covalent), $R_{\supseteq}$ (non-covalent), R_ (catalytic), R_ (mechanical)
Polarity	P	Directional character	P+ (acceptor), P- (donor), P_±ˆsym (symmetric), P_±ˆψ(pseudosymmetric), P_+- (donor-acceptor)
Fidelity	F	Thermodynamic reliability (I_net/ξ_CP)	$F_{\hbar}$ (high, ξ_CP ≤ 8.5 nats), F_ (medium, 8.5–11.0 nats), F_{ℓ} (low, >11.0 nats)
Kinetic Character	K	ΔG for constraint rearrangement	K_fast (<60 kJ/mol), K_mod (60–100), K_slow (>100), K_trap (pathway multiplicity), **K_MBL (many-body localization — disorder-frozen, ordinal below K_trap)**
Granularity	G	Correlation length / scale of control	G_ (local), G_ (mesoscale), G_ (global / non-local)
Interaction Grammar	Γ	Partner selection logic	Γ_∧ (AND), Γ_∨ (OR), Γ_→ (SEQUENTIAL), **Γ_ (DISSIPATIVE — irreversible loss)**; tiers: SPECIFIC / SELECTIVE / BROAD / **QUANTUM (superposition-preserving)**
Criticality Phase	Φ	G-D degeneracy condition	Φ_sub (subcritical), Φ_c (critical),

0.2.3 Grounding Validation (v2.1) — NEW

Fix 1: Registration Block on Grounding Warnings

- Synthons with ungrounded primitives can now be blocked from registration
- CLI flags: `-strict-grounding`, `-override-grounding`, `-override-reason`
- Audit trail logs all grounding overrides with human-provided justifications

Grounding Status Values:

- `full` — All primitives mechanistically grounded
- `partial` — Some primitives lack grounding (registered with warning)
- `override` — Registered despite grounding failure (with human override reason)
- `unverified` — No grounding check performed
- `flagged_for_review` — Marked for manual audit

0.3 The Relational Substrate

The eleven primitives are **relational operators**, not intrinsic attributes. Every element of the tuple describes a constraint between entities or a capacity for interaction — never a monadic property of a system in isolation.

- **F (Fidelity):** Reliability of *constraint satisfaction* relative to a competitor or binding partner. There is no "intrinsic F" — only F relative to a context. The CB[7] displacement series (6/6 experimental confirmations, `PRIMITIVE_PREDICTIONS.md` P-1) was predicted from the ordinal $F_\hbar$ `>` `F_eth` `>` F_ℓ alone, without knowing the intrinsic chemistry of the guests.
- **K (Kinetic):** A barrier to *rearrangement* between states — requires at least two states and an environment.
- **Γ (Grammar):** Partner selection logic by definition. Undefined without a partner.
- **Ω (TopoIndex):** Topological protection *against perturbations* — meaningless without an environment to be protected from.

The algebra enforces this. Every operation in `meet / join / tensor / path / lift / pipeline` requires at least one additional operand. There are no unary information generators. The mathematics cannot process "nothing but the object" — it requires a second operand (environment, partner, or target state) to compute anything new.

The algebra is asymmetric. `path(A->B) != path(B->A)`, the F-floor ratchet is directed, `lift` has no inverse. This is not a limitation; it is what makes predictions *directional* rather than merely topological. The framework encodes directed ordered constraints — structural realism in operational form.

Consequence: all confirmed predictions (P-1 through P-4 in `PRIMITIVE_PREDICTIONS.md`) were driven by ordinal comparisons, not absolute values. Intrinsic scalar properties were not required as inputs. The framework demonstrates that a classification scheme built entirely from directed relational ordinals is sufficient to generate correct quantitative predictions about physical systems.

0.4 Quick Start

0.4.1 Installation

```
# Using uv (recommended)
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt

# Or using pip
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

0.4.2 Basic Usage

```
from synthomnicon import (
    Synthon, Dimensionality, Topology, RecognitionMode,
    Polarity, Fidelity, Granularity, InteractionGrammar,
    KineticCharacter, CriticalityPhase,
)

# Create a synthon: carboxylic acid dimer (R^2(8) motif)
dimer = Synthon(
    name="carboxylic_acid_dimer",
    dimensionality=Dimensionality.MOLECULAR,        # D_and
    topology=Topology.CYCLIC_BOWTIE,                # T_
    recognition_mode=RecognitionMode.NON_COVALENT, # $R_{\
   supseteq}$ (H-bonding)
    polarity=Polarity.SELF_COMPLEMENTARY_PSEUDO,    # P_+/-^\
   psi (pseudosymmetric)
    fidelity=Fidelity.HIGH,                         # $F_{\hbar
   }$
    kinetic_character=KineticCharacter.FAST,        # K_fast
   (<60 kJ/mol)
    granularity=Granularity.LOCAL,                  # G_
    interaction_grammar=InteractionGrammar.SELECTIVE_AND,  # \
   Gamma_and(SELECTIVE)
    criticality_phase=CriticalityPhase.SUBCRITICAL,  # Phi_sub
    stoichiometry="1:1",                            # S ---
   homodimeric
)

print(f"Notation: {dimer.to_notation()}")
# Output: \langleD_wedge; T_bowtie; R_superset; P_pm_pseudo;
   F_hbar; K_fast; G_beth; Gamma_and(SELECTIVE); Phi_sub; 1:1\
   rangle
```

0.4.3 Computing Thermodynamic Efficiency (with Kinetic Fidelity)

```python
from synthomnicon.thermodynamics import compute_eta_CP,
    compute_kinetic_fidelity

# Compute \eta_CP and \xi_CP for the dimer (DeltaG(298K, gas)
    = -12 kJ/mol)
# Uses effective fidelity: F_effective = F_thermo x F_kinetic
result = compute_eta_CP(dimer, delta_g=-12.0,
    use_effective_fidelity=True)

print(f"\eta_CP (efficiency): {result.eta_CP:.2e}")
print(f"\xi_CP (inefficiency): {result.xi_CP:.2f} nats")
print(f"Waste factor: {result.waste_factor:.1e}x Landauer
    limit")

# Compute kinetic fidelity separately
k_char, f_kinetic = compute_kinetic_fidelity(delta_g_ddagger
    =45.0)
print(f"Kinetic Character: {k_char.value}, F_kinetic: {
    f_kinetic:.2f}")
```

0.4.4 Axiom-Guided LLM Design Agent (v0.4.5)

`synthon_agent.py` implements an autonomous relational design agent using the Anthropic SDK and the real SynthOmnicon Python API. The LLM cannot hallucinate impossible chemistry: every proposal is immediately rejected with a precise axiom trace.

```python
from synthon_agent import run_design

# One-liner: design an allosteric ABL inhibitor
history = run_design(
    goal="bivalent allosteric ABL inhibitor that closes T_perp
    to T_in topology gap from GNF-2",
    target="GNF-2",          # require a HotSwap path to this
    catalog entry
    phi_c_min=0.70,           # convergence threshold
    xi_cp_max=12.0,
    max_iterations=10,
)

# Or use the CLI:
# syncon design --goal "bivalent allosteric ABL inhibitor" --
    target GNF-2 --phi-c-min 0.70
```

For single-shot tool dispatch without the agent loop:

```python
from synthon_tool import SynthonTool

r = SynthonTool.distance("allosteric_domain", "active_site")
print(r.to_json())       # {"status": "ok", "distance": 5.4,
    ...}
```

```
r = SynthonTool.criticality("allosteric_domain", xi_r=8.5,
    xi_tau=1e10)
print(r.phi_c_score)     # 0.72

# Or CLI:
# syncon tool distance --a allosteric_domain --b active_site
# syncon tool criticality --name allosteric_domain --xi-r 8.5
    --xi-tau 1e10
```

0.5 CLI Quickstart (v2.2)

All commands are accessible via `syncon` or `synthomnicon`.

0.5.1 I(bits) Calibration Pipeline

```
# Run calibration on all three reference targets (vacuum)
syncon info-bits --calibrate

# With solvent correction (chloroform)
syncon info-bits --calibrate --solvent chloroform

# Run on a specific catalog entry
syncon info-bits carboxylic_acid_dimer --solvent THF -o report
    .json

# Output (example):
# I_recognition:   9.39 bits  (selectivity-determining)
# I_orientation:   4.58 bits  (overhead)
# I_net:           8.02 bits  (= I_rec - 0.3 x I_orient)
# I_total+solvent: 13.98 bits (chloroform DeltaS_solv = -28 J/
    mol.K)
```

Calibrated I(bits) ranges (Phase 1.1):

System	I_recognition	Expected
Carboxylic acid dimer ($R^2(8)$)	9.4 bits	9–10 bits
Triple H-bond DAD·ADA array	16.6 bits	14–18 bits
Proline aldol cycle (D_∞)	8.0 bits	6–9 bits/turn

Default I range: **6–11 bits** (domain-dependent). Replaces prior 4–6 bit heuristic.

0.5.2 Criticality Probe

```
# Single entry
syncon criticality-probe my_entry --xi-r 12.5 --xi-tau 1.5e6
```

```
# With degeneracy type and universality class hint
syncon criticality-probe my_entry --xi-r 13.8 --xi-tau 1e6 \
    --degeneracy-type -o probe.json

# Batch mode --- scan all Phi_c / T+D_inf entries
syncon criticality-probe --batch --export-candidates
   candidates.json

# Example output (batch):
#
#  Rank  Synthon            Phi_c Score  Candidacy
   Degeneracy
#
# |   1  | synthon_Varma... |     0.750 | Phi_c (critical)
    | logarithmic |
#
   \--------+--------------------+------------+----------------------+---
```

Degeneracy tiers (Phase 2.1):

Score	Tier	Description
0.0–0.30	`none`	G and D fully independent
0.30–0.60	`logarithmic`	Varma QXY: $\xi\_r \approx \ln i_\tau$ (weak Axiom 5)
0.60–0.85	`power-law`	Conventional QCP: $\xi\_r \sim i_\tau\hat{}(1/z)$, z finite
0.85–1.00	`collapse`	Direct G/D identity (strong Axiom 5)

0.5.3 Catalog Stoichiometry (Phase 3)

```
# Audit T entries for missing/inconsistent stoichiometry
syncon audit --pass 4 --dry-run

# Auto-backfill S='1:1' for P+/- entries (top 500 by default)
syncon catalog auto-stoichiometry --dry-run
syncon catalog auto-stoichiometry --limit 1000  # apply to
   1000 entries

# After backfill: run audit to verify reduction
syncon audit --pass 4 --auto-flag --dry-run
```

Stoichiometry rules (enforced in Pass 4):

- `T_ + S="1:1"` → require `P+/-` (sym or ψ)
- `T_ + S="n:m"` (n≠m) → require `(BROAD)` or `T_network`

- `T_ + no S` → auto-suggest `S="1:1"` if `P+/-` present, else flag manual

0.5.4 Cross-Domain Analogies

```
# Standard analogy search
syncon analogies carboxylic_acid_dimer --min-similarity 0.6

# Critical systems only (Phi_c score > 0.5)
syncon analogies my_entry --critical-only --min-similarity 0.5

# Stoichiometry-aware (strict S weight)
syncon analogies Boronate_Ester --stoichiometry-aware --
   exclude-flagged

# Combination
syncon analogies my_entry --critical-only --stoichiometry-
   aware \
    --exclude-flagged --limit 20
```

0.5.5 ξ_CP Calibrated Table

```
from synthomnicon.thermodynamics import calibrated_xi_cp_table

table = calibrated_xi_cp_table()
for name, entry in table.items():
    print(f"{name}: \xi_CP = {entry.xi_CP:.2f} +/- {entry.
   I_uncertainty_bits:.1f} bits -> tier={entry.fidelity_tier}"
   )
# Output:
# acetic_acid_homodimer:    \xi_CP = 6.66 nats [6.56--6.77] ->
   tier=HIGH
# triple_hbond_array:       \xi_CP = 7.65 nats [7.59--7.72] ->
   tier=HIGH
# sigma_hole_dimer:         \xi_CP = 7.59 nats [7.51--7.68] ->
   tier=HIGH
# proline_aldol_cycle:      \xi_CP = 9.21 nats [9.09--9.36] ->
   tier=MEDIUM
```

0.5.6 Criticality Analysis

```
from synthomnicon import analyze_criticality, Synthon, ...

# Create or load a synthon
synthon = Synthon(...)  # Your synthon here

# Analyze for criticality (scale-free behavior)
analysis = analyze_criticality(synthon)

print(f"Is Critical: {analysis.is_critical}")
print(f"Confidence: {analysis.confidence:.1%}")
```

```
print(f"Correlation Length: {analysis.correlation_length:.1f}"
    )
print(f"Recommendation: {analysis.recommendation}")
```

0.5.7 Using Domain Agents

```
from synthomnicon.domains.molecular import
    MolecularSynthonAgent
from synthomnicon.domains.supramolecular import
    SupramolecularSynthonAgent
from synthomnicon.domains.temporal import TemporalSynthonAgent

# Molecular domain
mol_agent = MolecularSynthonAgent()
synthons = mol_agent.list_molecular_synthons()

# Supramolecular domain - cooperativity analysis
supra_agent = SupramolecularSynthonAgent()
coop = supra_agent.compute_cooperativity_induction(3)  #
    Triple H-bond array
print(f"Superlinear: {coop['is_superlinear']}")

# Temporal domain - fidelity per cycle
temp_agent = TemporalSynthonAgent()
fidelity = temp_agent.compute_fidelity_per_cycle(k_cat=1.0,
    k_side=0.001)
print(f"F_cycle: {fidelity['f_cycle']:.4f}")
```

0.6 Architecture

```
synthomnicon/
|---- models.py           # Ten primitives and Synthon
    dataclass
|---- registry.py         # SynthonCatalog for storage and
    search
|---- constraints.py      # Constraint propagation engine
|---- thermodynamics.py   # \eta_CP and \xi_CP metrics
\---- domains/
     |---- molecular/      # Retrosynthetic analysis agents
     |---- supramolecular/ # Crystal packing, H-bond networks
     |---- temporal/       # Catalytic cycles, oscillatory
    reactions
     \---- hybrid/         # Multi-dimensional systems (MOFs)
```

0.7 Key Concepts

0.7.1 Synthons as Directed Relational Operators

A **synthon** is a directed relational operator: a minimal specification of constraint-enforcement capacity defined entirely by its interactions with a compatible context. No

primitive in the tuple $\langle D; T; R; P; F; K; G; \Gamma; \Phi; S; \Omega \rangle$ describes an intrinsic property of an isolated object. F is a competitive displacement rank — there is no $F_\hbar$ in isolation, only $F_\hbar$ relative to a specified competitor set. K is a barrier relative to an environmental driving force. Γ is partner-selection logic that presupposes a partner. A tuple without a context encodes interaction affordances — what constraints it can enforce, against which partners, at what scale — never the constitution of a substance.

```python
from synthomnicon.constraints import ConstraintEngine

engine = ConstraintEngine()

# Check compatibility between two synthons
report = engine.check_pair_compatibility(synthon_a, synthon_b)
print(f"Compatible: {report.is_compatible}")
print(f"Details: {report.details}")
```

0.7.2 Cross-Domain Analogy

The unified notation enables searching for analogies across domains:

```python
from synthomnicon.registry import global_catalog

# Find temporal synthons analogous to a supramolecular synthon
supra_synthon = global_catalog.search_by_domain("
   supramolecular")[0]
temporal_analogs = global_catalog.find_cross_domain_analogs(
    supra_synthon,
    target_domain="temporal",
)
```

0.7.3 Constraint Propagation Efficiency

The metrics η_CP and ξ_CP quantify how effectively a synthon converts energy into information:

- η**_CP** = (I × F) / (ΔG / E_bit) — efficiency (0 to 1)
- ξ**_CP** = -ln(η_CP) — inefficiency in nats

Calibrated reference values (ΔG(298 K) basis, QUANTSYNTHONICON.md Section VI):

System	ξ_CP (nats)	Uncertainty	Tier
Acetic acid homodimer	6.66	[6.56–6.77]	HIGH
σ-Hole dimer (CFI·NMe)	7.59	[7.51–7.68]	HIGH
Triple H-bond array	7.65	[7.59–7.72]	HIGH
Proline aldol cycle	9.21	[9.09–9.36]	MEDIUM

0.8 Tuple Algebra (v0.3.2)

Six new commands implement the full compositional algebra over the eleven-tuple space (`synthomnicon/algebra.py`).

0.8.1 Lattice Operations

```
# Meet (greatest lower bound) --- identify shared design floor
syncon meet
   Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon
   nitroso_radical_redox_synthon_pair

# Join (least upper bound) --- identify minimal common ceiling
syncon join
   Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon
   nitroso_radical_redox_synthon_pair
```

Output: side-by-side tuple diff with CONFLICT markers on incompatible primitives and the result tuple in unified notation.

0.8.2 Path Search

```
# BFS over valid-swap graph --- find multi-hop redesign
   sequence
syncon path SOURCE DESTINATION [--max-hops 5] [--xi-tolerance
   2.0]

# Example
syncon path
   Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon
   nitroso_radical_redox_synthon_pair
```

Finds the shortest path in the directed HotSwap graph. Restricted to the same {D, T} cluster. Reports per-hop $\Delta\xi$_CP and cumulative cost. Asymmetric: `path(A->B)` may exist when `path(B->A)` does not (F-floor enforcement).

0.8.3 Tensor Product

```
# Ensemble prediction --- effective tuple of a two-component
   assembly
syncon tensor SYNTHON_A SYNTHON_B [--lambda 0.5]
```

Computes $s_1 \otimes s_2$: F→min, K→min, G→max, T→promote, Φ_c propagates, ξ_ens = $\xi+\xi-\lambda\cdot$I(s;s).

0.8.4 Natural Transformations (Lift)

```
# Migrate a synthon across domains
syncon lift SYNTHON temporal     # D_and->D_inf, R->R_, K_fast
   ->K_mod, \Gamma->SEQUENTIAL
syncon lift SYNTHON spatial      # D_and->D_, T->T_ if
   applicable, G_->G_
```

```
syncon lift SYNTHON critical     # Phi_sub->Phi_c (requires F>=
   $F_{\hbar}$, else BLOCKED)
syncon lift SYNTHON molecular    # forgetful functor: D_inf->
   D_and, R_->$R_{\subseteq}$, \Gamma_->->\Gamma_and
```

0.8.5 Composable Design Pipeline

```
# Chain operations with automatic \xi_CP threading and fail-
   fast on blocks
syncon pipeline START \
  --step meet:OTHER \
  --step lift:temporal \
  --step path:TARGET:xi_tolerance=1.5 \
  --step tensor:THIRD
```

Writer+Maybe monad: accumulates $\Delta\xi$_CP across all steps; BLOCKED steps are logged and the pipeline continues from the last valid state. Prints a full trace with step-by-step tuple diffs and cumulative costs.

0.8.6 Decomposition Algebra

```
# Cofactor --- given composite ~= tensor(A, B), reconstruct B
   given A
syncon cofactor COMPOSITE KNOWN_FACTOR

# Birkhoff principal decomposition --- join-irreducible atoms
syncon decomp SYNTHON_NAME

# Project onto primitive subspace
syncon project SYNTHON_NAME --keys Phi,Omega,T

# Heyting pseudocomplement --- largest synthon meeting given
   constraint
syncon complement SYNTHON_NAME --projection Phi,Omega --
   reference TARGET
```

`cofactor` reports per-axis roles: BOTTLENECK, CONTRIBUTOR, EXPLAINED, PASSTHROUGH, or CONFLICT. `principal_decomp` factors into named atoms (`atom[F=...]`, `atom[K=...]`, `atom[G=...]`, `skeleton(...)`) that compose back to the original under join. See `TENSOR_OPS_DEMO.py` ğ7 for three worked examples.

0.8.7 Phase Detection

```
# Compute pairwise distances, Ward clustering, and MDS
   projection over a synthon set
syncon phase-diagram
       # default: all quantum synthons
syncon phase-diagram spin_singlet kitaev_chain_majorana fqh
      # named subset
```

```
syncon phase-diagram --save phase_map.png
      # render figure
syncon phase-diagram --text-only
       # skip matplotlib
syncon phase-diagram --format json
      # machine-readable output
```

Detects phase boundaries from syntax alone: no physics is input beyond primitive assignments. The primary boundary in the 8-synthon quantum catalog falls at d $\approx$ 9.52 (extended topological matter / quantum particles), reproduced by Ward hierarchical clustering on the tuple distance matrix.

0.8.8 Python API

```
from synthomnicon.algebra import (
    meet, join, find_path, tensor, criticality_lift,
    lift_to_temporal, lift_to_spatial, project_to_molecular,
    DesignPipeline,
    cofactor, principal_decomp, project, complement_rel,
)

# Lattice
result = meet(s1, s2)         # -> LatticeResult
result = join(s1, s2)         # -> LatticeResult
print(result.to_notation())  # unified notation of result
   tuple

# Path search
path = find_path(src, dst, catalog, max_hops=5, xi_tolerance
   =2.0)  # -> PathResult

# Tensor product
ens = tensor(s1, s2, lambda_=0.5)   # -> TensorResult
print(ens.xi_ensemble)               # effective \xi_CP of
   assembly

# Natural transformations
lifted = lift_to_temporal(s)         # -> LiftResult
lifted = criticality_lift(s)         # -> LiftResult (BLOCKED
   if F < $F_{\hbar}$)

# Composable pipeline (Writer+Maybe monad)
result = (
    DesignPipeline
    .start(synthon)
    .meet(other)
    .join(second)
    .lift("critical")
    .path("target_name")
    .result()                        # -> PipelineResult with
   full trace
```

```
)
result.print_trace()

# Decomposition algebra
cf = cofactor(composite, known_factor)    # -> CofactorResult;
   cf.dimensions lists axis roles
atoms = principal_decomp(s)               # -> list[Synthon];
   join-irreducible Birkhoff atoms
proj = project(s, ["Phi", "Omega", "T"]) # -> Synthon
   projected onto primitive subspace
comp = complement_rel(s, proj, ref)       # -> ComplementResult
   ; comp.satisfied, comp.notes
```

0.9 Protocol Suite (v0.3.0)

Four new analysis protocols added in v0.3.0:

Protocol	Module	CLI	Description
SYNTHONIC_PERTURBATION	`perturbation.py`	`syncon perturb sweep`	Primitive Jacobian — $\Delta\xi$_CP sensitivity for all primitives ± 1 tier
SYNTHONIC_TRAJECTORY	`trajectory.py`	`syncon trajectory validate`	D_∞ cycle validation: Axiom 6, continuity, kinetic traps, Varma probe
SYNTHONIC_ENSEMBLER	`ensembler.py`	`syncon ensemble check`	N×N pairwise compatibility + emergent property detection
SYNTHONIC_RETROISIGN	`retrodesign.py`	`syncon retrodesign`	Axiom-pruned retrosynthetic decomposition tree

0.9.1 Quick Protocol Examples

```
# Perturbation sweep --- which primitive most affects \xi_CP?
syncon perturb sweep carboxylic_acid_dimer --delta-g -12.0

# Trajectory --- validate a D_inf catalytic cycle
syncon trajectory validate --steps enamine,c_c_bond,hydrolysis
    --reset hydrolysis

# Ensembler --- check multi-synthon compatibility
syncon ensemble check --components carboxylic_acid_dimer,
   proline_aldol_cycle
```

```
# Retrodesign --- decompose a target into valid sub-tuples
syncon retrodesign carboxylic_acid_dimer --max-depth 3 --prune
   -axioms 1,2,4,6
```

See `examples/demo_protocols.py` for a full integration demo using calibrated reference values.

0.10 Examples

Run the demo scripts to see the framework in action:

```
# Core framework examples
python examples/synthomnicon_examples.py

# Four-protocol suite demo (v0.3.0)
python examples/demo_protocols.py
```

`demo_protocols.py` demonstrates:

1. **Perturbation** — primitive Jacobian sweep on `carboxylic_acid_dimer` (ΔG = -12.0 kJ/mol)
2. **Trajectory** — three-step proline aldol cycle validation with Axiom 6 reset verification
3. **Ensembler** — three-component ensemble (molecular + temporal + mechanical) with emergent property scan
4. **Retrodesign** — retrosynthetic decomposition with axiom pruning (Axioms 1, 2, 4, 6)

`synthomnicon_examples.py` demonstrates:

1. Creating synthons with eleven primitives
2. Computing thermodynamic efficiency
3. Catalog storage and search
4. Cross-domain analogy finding
5. Constraint compatibility checking
6. Domain-specific agent usage

0.11 Testing

Run the integration tests:

```
python test_integration.py
```

Tests cover:

- Synthon models and primitives
- Catalog registration and search
- Constraint propagation engine
- Thermodynamics (η_CP, ξ_CP)

- Domain agents
- Framework integration

0.12 API Reference

0.12.1 Core Classes

Synthon

```
Synthon(
    name: str,
    dimensionality: Dimensionality,
    topology: Topology,
    recognition_mode: RecognitionMode,
    polarity: Polarity,
    fidelity: Fidelity,
    granularity: Granularity,
    interaction_grammar: InteractionGrammar,
    description: str = "",
    metadata: Dict[str, Any] = {},
)
```

Methods:

- `to_notation()` → str: Unified notation string
- `to_json()` → str: JSON serialization
- `is_compatible_with(other)` → Dict: Compatibility check
- `constraint_strength` → float: Overall constraint strength (0-1)

SynthonCatalog

```
catalog = SynthonCatalog(name="my_catalog")
catalog.register(synthon)
results = catalog.search(fidelity=Fidelity.HIGH)
analogs = catalog.find_cross_domain_analogs(synthon, "temporal
    ")
```

0.12.2 Thermodynamics Functions

```
compute_eta_CP(synthon, delta_g, information_gain=None)
compute_xi_CP(synthon, delta_g, information_gain=None)
benchmark_against_landauer(synthon, delta_g)
compare_efficiencies([(synthon1, dG1), (synthon2, dG2)])
```

0.12.3 Domain Agents

```
# Molecular
mol_agent = MolecularSynthonAgent()
mol_agent.analyze_reaction_center(smiles)
mol_agent.get_synthon_polarity(smiles, functional_groups)

# Supramolecular
supra_agent = SupramolecularSynthonAgent()
```

```
supra_agent.analyze_hydrogen_bond_network(cif_path)
supra_agent.compute_cooperativity_induction(n_hbonds)

# Temporal
temp_agent = TemporalSynthonAgent()
temp_agent.analyze_reaction_cycle(cycle_name, catalyst)
temp_agent.compute_fidelity_per_cycle(k_cat, k_side)
```

0.13 Integration with AjintK Framework

SynthOmnicon integrates with the AjintK multi-agent framework:

```
from framework import BaseAgent
from synthomnicon import Synthon, compute_eta_CP

class SynthonAnalysisAgent(BaseAgent):
    async def run(self, task: str, context=None):
        # Use synthomnicon for analysis
        synthon = create_synthon_from_task(task)
        efficiency = compute_eta_CP(synthon, delta_g=-50.0)

        return {
            "status": "success",
            "findings": f"\xi_CP = {efficiency.xi_CP:.2f} nats
  ",
            "metadata": {"eta_CP": efficiency.eta_CP},
        }
```

0.14 AI-Powered Synthon Generation

The `SynthonGeneratorAgent` uses LLM reasoning to automatically generate synthons from natural language descriptions or SMILES strings:

```
import asyncio
from agents.synthon_generator_agent import
   SynthonGeneratorAgent, generate_synthon
from synthomnicon.provider_config import build_agent_config

async def main():
    # Use config-driven defaults (model=None uses provider
   default)
    config = build_agent_config(provider="anthropic", model=
   None)
    agent = SynthonGeneratorAgent(config)

    # Generate from natural language
    result = await agent.generate_from_description(
        "carboxylic acid dimer with cyclic hydrogen bonding",
        delta_g=-12.0,  # Optional: for thermodynamic analysis
    )
```

```
    print(f"Generated: {result.synthon.name}")
    print(f"Notation: {result.synthon.to_notation()}")
    print(f"Confidence: {result.confidence:.1%}")
    print(f"Reasoning: {result.reasoning}")

    # Generate from SMILES
    result = await agent.generate_from_smiles("CC(=O)O", name=
   "acetic_acid")

    # Convenience function
    result = await generate_synthon(
        "DNA adenine-thymine base pair",
        provider="qwen",
        model="qwen3-max"
    )

asyncio.run(main())
```

0.14.1 CLI Commands for AI Generation

The CLI is accessible via both `synthomnicon` and the short alias `syncon`:

```
# Standard synthon generation
synthomnicon generate "carboxylic acid dimer" --delta-g -12.0
syncon generate "carboxylic acid dimer" --delta-g -12.0

# Axiom-guided generation (validates 5 composition axioms) ---
    NEW in v2.0
syncon generate "carboxylic acid dimer" --axiom-guided
syncon generate "DNA base pair" -a --provider deepseek

# Grounding-controlled registration --- NEW in v2.1.3
syncon generate "..." --strict-grounding              # Block
   on grounding failure
syncon generate "..." --override-grounding --override-reason "
   novel system"
syncon generate "..." --speculative                   #
   Register in speculative domain

# Generate from SMILES
syncon generate-smiles "CC(=O)O" --name acetic_acid

# Compare synthons
syncon compare carboxylic_acid_dimer adenine_thymine_pair

# View catalog as tree
syncon catalog tree --domain molecular

# Export catalog
syncon export --format json --output synthons.json
```

```
# Criticality analysis --- NEW in v2.0
syncon criticality carboxylic_acid_dimer
syncon criticality --all --min-confidence 0.7

# Catalog audit --- NEW in v2.1.3 (Fix 6)
syncon audit --axiom 6                                    # D_inf
   closed-cycle audit
syncon audit --axiom 7 --auto-flag                        # T_
   closing-bond audit + flag
syncon audit --status unverified --dry-run                #
   Preview unverified entries
```

0.14.2 Agent Framework CLI

The AjintK agent framework is accessible via the `agents` subcommand:

```
# List available agents
syncon agents list

# Run SynthonGeneratorAgent from CLI
syncon agents run -d "carboxylic acid dimer" -g -12.0
syncon agents run --provider qwen --model qwen3-max -d "DNA
   base pair"

# Run AxiomGuidedGeneratorAgent --- NEW in v2.0
syncon agents run -d "carboxylic acid dimer" -a  # --axiom-
   guided

# Generate from SMILES via agent
syncon agents from-smiles "CC(=O)O" --name acetic_acid
syncon agents from-smiles "CC(=O)O" -o result.json

# Autonomous discovery
syncon agents discover --cycles 50 --focus "halogen bonding"
```

0.15 Further Reading

1. **QUANTSYNTHONICON.md** — Theoretical foundation with all eight transformations
2. **METHODOLOGY.md** — Framework design philosophy
3. **QUICKSTART.md** — AjintK framework quick start guide
4. **AGENTS.md** — Agent development guide

0.16 Recent Changes

- **March 18, 2026 — v0.4.5: Programmable matter domain · LLM tool layer · `syncon tool` / `syncon design`**:
  - **Programmable matter domain** (`PROGRAMMABLE_MATTER.md`, `programmable_matter_test` `programmable_matter_tests2.py`): 11 PM synthon encodings spanning DNA

strand displacement, colloidal crystals, biomolecular condensates, shape-memory polymers, and liquid-crystal elastomers. Full algebra suite: pairwise distance matrix, meet/path algebra, Primitive Jacobian (/), tensor products (Φ_c propagation), DesignPipeline monad (4 routes), programmability lattice (dynamic floor = Φ_c theorem). 10 formal predictions P-38 through P-47 added to `PRIMITIVE_PREDICTIONS.md`.

- **`synthon_tool.py`** — SynthonTool dispatch layer and `SYNTHON_TOOL_SCHEMA` (Anthropic/OpenAI format). `SynthonTool.dispatch(operation, **kwargs)` routes all 7 operations (`validate`, `criticality`, `path`, `analogies`, `distance`, `meet`, `generate`) to the live Python API. `ToolResponse` dataclass serialises to JSON for LLM tool-result injection. The LLM cannot hallucinate impossible chemistry: every proposal is rejected with a precise axiom trace if it fails.
- **`synthon_agent.py`** — Autonomous relational design agent. `SynthonDesignAgent.run(max_` loop: LLM proposes encoding → tool dispatch validates → Φ_c probe → HotSwap path → convergence check (Φ_c ≥ threshold AND ξ_CP ≤ threshold). Full Anthropic SDK message threading. `ConvergenceCriteria` dataclass configures stopping conditions. `run_design(goal, target, phi_c_min, xi_cp_max)` one-liner convenience function.
- **`syncon tool` CLI command**: single-shot SynthonTool dispatch from the command line. All 7 operations with `-format text|json` output. Example: `syncon tool distance -a allosteric_domain -b active_site`.
- **`syncon design` CLI command**: autonomous LLM design agent from the command line. Options: `-goal`, `-target`, `-phi-c-min`, `-xi-cp-max`, `-max-iterations`, `-model`, `-quiet`, `-output`. Prints Rich iteration table on completion.
- **`SYNTHONICON.md ğX`** expanded from 11 to ~100 lines: X.1 Path algebra theorem, X.2 Eleven encodings table, X.3 F–K programmability quadrant, X.4 Φ_c as global programmability, X.5 programmability lattice (floor = Φ_c theorem), X.6 cross-domain analogies (condensate ≈ allosteric d=2.50, colloidal ≈ TI d=2.80).

- **March 17, 2026 — v0.4.1: Molecular domain catalog · Design suite at 100% execution · 9 canonical synthons**:
  - **`synthomnicon/domains/molecular/__init__.py`** — `register_molecular_synthons()` programmatically registers nine canonical molecular and supramolecular synthons at import time, surviving catalog JSON resets. Auto-invoked in `__init__.py` alongside the cross-domain and quantum registration chains.
  - **Nine new catalog entries**: `nitroso_radical_redox_synthon_pair` (D_∞, T_, $F_\hbar$ — Frank-model Factor 7 candidate); `amide_dimer` (D_∧, T_, F_eth — H-bond dimer lattice floor); four cage/bowl supramolecular hosts (`cavitand`, `calixarene`, `crown_ether`, `cryptand`, `cucurbituril`) spanning F_eth–$F_\hbar$; two ionic fragments (`CH`, `CH`) for nucleophile/electrophile retrodesign.
  - **Design suite: 18/20 scripts pass** — all `[ERROR]` (missing synthon) failures resolved. 2 intentional F-floor pedagogical demonstrations (designs 01 and 04) remain blocked as expected. Key algebra results confirmed: F = min(F, F) tensor bottleneck; T_cage ⊗T_cage → T_cage topology promotion; mplus fallback chain; Axiom 6 propagation through D_∞ hybrid; MI discount of 7–8.6 nat for complementary-charge tensors.

  - **Validation row V-6** added to Section XIV of SYNTHONICON.md (algebra correctness suite).
  - **SYNTHONICON_LANG.md** fully fleshed out: Phase 3e section added (PhaseDiagram module, universality track prediction, MDS embedding, Floquet/gauge-theory stress tests); File Index and Open Questions updated.
- **March 16, 2026 — v0.4.0: Quantum primitive extensions · 11th primitive Ω · K_MBL · T_braid · Factor 8 · distance command**:
  - **Ω (TopoIndex) — 11th primitive**: `Omega_0` (trivial) / `Omega_Z` (Kitaev chain, SSH) / `Omega_Z2` (topological insulators) / `Omega_C` (Chern/IQH) / `Omega_NA` (non-abelian anyons, ν=5/2 FQH). Integrated into `tuple_distance`, `meet`, `join`, `tensor`, `compare`, `analyze`, perturbation sweep. All existing catalog entries default to `None` (trivial) — fully backward-compatible.
  - **T_braid**: new topology value for anyonic/braided exchange statistics (fractional QHE, Kitaev honeycomb). Placed at complexity 4, below T_cage. `tensor(T_braid, T_braid) -> T_braid` (anyonic statistics preserved under composition). `T_braid  T_linear = ⊥` — the unresolvable conflict that motivated the primitive.
  - **K_MBL**: new kinetic character for many-body localization — disorder-frozen, not barrier-limited. Ordinal position 0 (below K_trap at 1), reflecting that MBL is more kinetically arrested than any energy-barrier trap. `meet(K_MBL, K_fast) -> K_MBL`. `K_trap -> K_MBL` perturbation shift on spin singlet: $\Delta\xi$= +2.30 nats, sensitivity HIGH.
  - **Γ_ (DISSIPATIVE)**: new grammar operator for irreversible information loss (Lindblad dynamics, quantum Zeno). Four new `InteractionGrammar` members: `SPECIFIC/SELECTIVE/BROAD_DISSIPATIVE`.
  - **QUANTUM grammar tier**: `QUANTUM_AND/OR/SEQ/DISSIPATIVE` — Toffoli-gate semantics preserving superposition, distinct from classical AND.
  - **Factor 8 in Varma probe**: quantum criticality fingerprint — `G_aleph + F_hbar + K_trap + notD_inf`. Maps to transverse-field Ising at $h = h_c$, heavy fermions (CeCuAu), quantum dots at charge degeneracy. Spin singlet scores 0.20, universality class "quantum_criticality (TFI/heavy_fermion)". Falsifiable prediction: $\chi(T0) \sim T^{-\gamma}$.
  - **`syncon distance` command**: weighted quasi-metric with symmetric and directed (HotSwap-asymmetric) modes. Full 10×10 quantum pairwise matrix computed — proton↔electron closest (d=1.50, ~0 asymmetry); proton↔spin most distant (d=6.00).
  - **All algebra commands live in `syncon`**: `meet`, `join`, `path`, `tensor`, `lift`, `distance`, `pipeline` were registered with `main` but not `syncon_alias` — now wired.
- **March 16, 2026 — v0.3.8: Quantum domain · Axiom 1 as classical boundary detector · G_aleph first appearance**:
  - **Quantum particle series (5 systems)**: entangled photon, proton, electron, spin, qubit encoded using the framework with no chemical template. Each vague input yielded a distinct, physically defensible tuple from primitive definitions alone.
  - **$G_\aleph$ first appearance in catalog**: assigned to spin and qubit only — encoding

quantum non-locality (Bell inequality; constraint propagates without spatial attenuation). Every classical system in the catalog is $G_{\sqsupset}$ or $G_{\urcorner}$.

- **Axiom 1 as classical boundary detector**: Axiom 1 flagged $T_{\bowtie} + P_{\pm}^{\text{sym}} + F_{\ell}$ for entangled spin — a violation irresolvable in the classical frame. Root cause: F measures constraint *reliability*, not Shannon channel capacity (no-communication theorem). Spin singlet fires with 100% reliability $\rightarrow F_{\hbar}$, not F_{ℓ}. The violation pattern is a diagnostic for quantum systems, not a falsification of the axiom.
- **Corrected spin encoding**: $F_{\ell} \rightarrow F_{\hbar}$, $K_{\text{fast}} \rightarrow K_{\text{trap}}$ — spin singlet is permanent (like photon), not dynamically exchangeable. Catalog updated.
- K **encodes kinetic regime correctly** without domain-specific training: photon (K_{trap}), proton (K_{fast}), electron (K_{fast}), spin (K_{trap}), qubit (K_{fast}). All five physically justified.
- SYNTHONICON.md updated: Axiom 1 quantum boundary condition note in ğIV; full quantum domain subsection and 5-particle comparison table in ğVII; V-5 validation row in ğXIV.

- **March 16, 2026 — v0.3.7: Ice polymorph catalog · T_∈ integration · Domain-agnostic prompts**:
  - **Ice polymorph family** (13 phases, ice Ih–XIX): full catalog encoding using $T_{\in}$ sub-labels. Four sub-labels resolve all identical-tuple collisions: $T_{\in}$(hex) (Ih/Ic/XI), $T_{\in}$(mixed) (III/IV/V/IX), $T_{\in}$(×2) (VI/VII/VIII), $T_{\in}$(sym) (X).
  - K_{fast} **as causal primitive (ice VI)**: validated against dielectric relaxation data (Yamane 2021). K_{fast} is the causal primitive enabling multiple ordering landscapes (ice XV at ~1 GPa, ice XIX at >1.5 GPa). The $K_{\text{fast}} \rightarrow K_{\text{slow}}$ flip encodes the ordering transition as a single primitive change.
  - G **encodes pressure-dependent ordering correlation length**: ice XV ($G_{\urcorner}$, mesoscale cooperative ordering) vs. ice XIX ($G_{\sqsupset}$, local — shorter O-O distances at >1.5 GPa constrain local geometry). Falsifiable prediction against future neutron correlation-length data.
  - **T_∈ sub-label integration** across `constraints.py` (cooperativity: INTERP=3.0, SYM=2.8, HEX=2.5, MIXED=2.3), `algebra.py` (_T_ORD tier 4 + cross-sub-type join → generic NETWORK), `ensembler.py` (n_network count + has_global_grammar), `perturbation.py` (_TOPOLOGY_TIERS), `cli.py` (Pass 4 has_network_topo).
  - **Domain-agnostic agent prompts**: both `SynthonGeneratorAgent` and `AxiomGuidedGeneratorAgent` reframed from "computational chemist / chemical accuracy" to information-theoretic reasoning. Primitive descriptions now lead with ξ_CP/I_net (F), ΔG (K), and correlation length (G). Reference examples expanded to include ice polymorphs alongside molecular systems.
  - **`syncon compare` shows all 10 primitives**: K (Kinetic Character), Φ (Criticality), and S (Stoichiometry) were missing from the comparison table — now included.
  - Catalog fixes: ice I_C F_ell→F_eth (same H-bond geometry as Ih); ice VI D_wedge→D_triangle (bulk crystal, not molecular).
- **March 16, 2026 — v0.3.5: Phase 3a DSL · Agent Fixes · Topology Symbols**:
  - ∗ **`.syn` YAML DSL evaluator** (`synthomnicon/syn_runner.py`, NEW

~380 lines):`SynScript` class: compiles YAML design programs into `SynthonM` pipelines; `join`, `meet`, `tensor`, `lift`, `path`, `assert`, `bind`, `or` step types
    - `or:` step → `strategy_or(branch_A, branch_B)` — MonadPlus fallback semantics
    - Post-hoc `output:` `assert:` block for WriterT-level cost/context checks (`total_delta_xi`, `steps`, `criticality_ok`)
    - Safe predicate grammar: no `eval`; 7 predicate forms dispatched explicitly
  - **`syncon run` CLI command** (`synthomnicon/cli.py`): `-format text|json`, `-save PATH`, `-dry-run`; runs `.syn` scripts from the command line
  - – * **Agent primitive awareness fixed** (both `agents/axiom_guided_generator.py` and `agents/synthon_generator_agent.py`):Updated to **ten primitives** (was "seven"/"nine") in all task instruction and system prompt locations
    - `criticality_phase` parser now defaults to `Phi_sub` instead of silently returning `None`
    - Topology block updated to list all 7 values with explicit symbols
  - **New topology symbols**: $T_{|}$ (LINEAR), $T_{\perp}$ (BRANCHED), $T_{\in}$ (NETWORK) — registered in `Topology.from_symbol()`, documented in SYNTHONICON.md ğII
  - **`to_notation()` always includes** Φ: defaults to `Phi_sub` when `criticality_phase` is unset; S appended as 10th position
  - **`algebra.py` `is_degenerate` fix**: `CriticalityPhase.is_degenerate` is a `@property` — removed erroneous `()` call at 4 sites
  - `__init__.py` version: `0.3.5`; new exports: `SynthonM`, `SynScript`, `SynParseError`, `UnknownAssertion`, `run_syn_file`, all monad combinators
- **March 16, 2026 — v0.3.3: Experimental Validation + Factor 7 + F_ell Activation**:
  - **CB[7] competitive displacement — 6/6 HotSwap validation** (Kim JACS 2001; Assaf & Nau CSR 2015):
    - Three catalog entries: `CB7_ferrocene_ammonium_complex` ($F_\hbar$, Ka=3×10ź^2), `CB7_adamantane_ammonium_complex` (F_, Ka=4×10), `CB7_DABCO_complex` (F_ℓ, Ka=2×10)
    - All 6 directional HotSwap predictions match experiment exactly from ordinal F ranking alone
    - Confirms the F-floor asymmetric ratchet: Fc displaces Ad and DABCO; Ad displaces DABCO but not Fc; DABCO displaces neither
    - Activates the F_ℓ **(LOW) tier** as first populated catalog entry — Ka threshold < ~10 Mź (ΔG ≈ −40 kJ/mol)
  - **Factor 7 — Frank-model classical bifurcation** (`synthomnicon/varma_probe.py`):
    - New scoring factor in `score_phi_c_candidacy()` (weight 0.25)
    - Fires when all four Frank co-requisites present: D_∞ + T_ + P_directional + $F_\hbar$
    - Detects pitchfork bifurcation at ee = 0 — universality class distinct from Varma QXY and steric-cliff mechanisms
  - **Three Varma probe scoring mechanisms now operational:**
    - Factors 1–5: Varma QXY structural heuristics

    - ∗ Factor 6: Steric-cliff proxy (db24c8, proxy_degeneracy_strength ≥ 0.50)
    - ∗ Factor 7: Frank-model classical bifurcation (D_∞ + T_ + P_DA + $F_\hbar$ co-present)
  - **Experimental validation — three-system discrimination:**

System	Score	Candidacy	Mechanism
Soai autocatalytic cycle	**0.920**	approaching Φ_c	Frank-model bifurcation (Factor 7)
DB24C8 pseudorotaxane	**0.461**	approaching Φ_c	Steric-cliff proxy (Factor 6)
Proline-aldol cycle	**0.380**	Φ_sub	None (ratio = 0.189, subcritical)

  - **Soai reaction catalog entry** (`soai_pyrimidyl_autocatalytic_cycle`): D_∞, T_, R_, P_DA, $F_\hbar$, K_mod, G_, Γ_→(SPECIFIC), Φ_sub, 1:1. ΔG = 62.3 kJ/mol. ξ_r = 15, i_τ = 7.2×10ź, ratio = 0.94. Full grounding (Soai 1995, Gridnev 2010, Shibata 2009).
  - **Proline-aldol Varma probe**: ξ_r = 6.2, i_τ = 1.8×10ź (ω_c = 10ź^2 sź), ratio = 0.189 → Φ_sub confirmed (Blackmond RPKA 2004; Houk/List 2004)
  - **db24c8 full grounding** + `_has_phi_c_candidacy()` batch filter + SYNTHONICON.md section reorder (v0.3.3 base)
  - `__init__.py` version: `0.3.3`
- **March 16, 2026 — v0.3.2: Tuple Algebra + Compositional Design Language**:
  - ∗ **`synthomnicon/algebra.py`** (NEW ~1200 lines):**Quasi-metric**: `tuple_distance(s1, s2, weights, symmetric)` — weighted Hamming quasi-metric over the eleven-tuple
  - ∗ **Lattice**: `meet(s1, s2)`, `join(s1, s2)` — componentwise min/max on ordered primitives (F,K,G), CONFLICT sentinel on categorical mismatch; Φ_c dominates in both operations
  - ∗ **Path algebra**: `find_path(src, dst, catalog, max_hops, xi_tolerance)` — BFS over valid-swap directed graph (same {D,T} cluster), accumulates Δξ_CP
  - ∗ **Tensor product**: `tensor(s1, s2, lambda_)` — ensemble prediction: F→min, K→min, G→max, Φ_c propagates, ξ_ens = ξ+ξ−λ·I(s;s)
  - ∗ **Natural transformations**: `lift_to_temporal`, `lift_to_spatial`, `criticality_lift` (F≥$F_\hbar$ gate), `project_to_molecular`
  - ∗ **DesignPipeline**: Writer+Maybe monad — chains meet/join/tensor/lift/-path with automatic ξ_CP threading and fail-fast logging
  - ∗ **Six new CLI commands** (`synthomnicon/cli.py`):`syncon meet S1 S2` — lattice meet with CONFLICT reporting
  - ∗ `syncon join S1 S2` — lattice join
  - ∗ `syncon path SRC DST [-max-hops N] [-xi-tolerance F]` — BFS path search
  - ∗ `syncon tensor S1 S2 [-lambda F]` — ensemble prediction

    - * `syncon lift NAME (temporal|spatial|critical|molecular)` — natural transformation with side-by-side diff
    - * `syncon pipeline START -step op:arg[:key=val] ...` — composable pipeline
  - * **Grounding patches** (`~/.synthomnicon/catalog.json`):Full grounding applied to `nitroso_radical_redox_synthon_pair`, `Dithiadiazolyl_Phthalocyanine` and `Cucurbituril-Viologen Mechanically Interlocked Synthon`
    - * Verified: `syncon hotswap Dithia... nitroso...` → APPROVED; `syncon hotswap db24c8... Cucurbituril...` → APPROVED
  - **SYNTHONICON.md**: New Section XIV — Tuple Algebra and Compositional Design (metric space, lattice, path algebra, tensor, natural transformations, DesignPipeline, monad stack)
  - `__init__.py` version bumped to `0.3.2`
- **March 15, 2026 — v0.3.0: Four-Protocol Suite Implementation**:
  - * **SYNTHONIC_PERTURBATION** (`synthomnicon/perturbation.py`):`Perturbati` — full primitive Jacobian, $\Delta\xi$_CP for ±1 tier across all 8 perturbable primitives
    - * `PerturbationEngine.fault_injection()` — single-point-of-failure analysis
    - * `PerturbationEngine.find_path_to_target()` — minimum-step path to a target ξ_CP
    - * `PrimitiveJacobian` dataclass: `baseline_xi_CP`, `results`, `most_sensitive`, `critical_primitives`
    - * Sensitivity labels: `CRITICAL` ($\Delta\geq3.0$ nats), `HIGH` (≥1.5), `MEDIUM` (≥0.5), `LOW`
    - * CLI: `syncon perturb sweep <name> -delta-g <float> [-format json]`
    - * CLI: `syncon perturb fault-injection <name> -delta-g <float>`
    - * CLI: `syncon perturb pathfind <name> -delta-g <float> -target <float> -optimize F,K`
  - * **SYNTHONIC_TRAJECTORY** (`synthomnicon/trajectory.py`):`TemporalSyntho` — validate D_∞ cycles as step sequences
    - * Three continuity checks: S mass balance, Axiom 4 (D_∞ or R_), K_trap/ΔG>100 detection
    - * Axiom 6 reset: explicit `is_reset=True` flag or description keyword matching
    - * Varma probe integration: per-step degeneracy scoring and Φ_c candidacy
    - * `TrajectoryValidationResult`: `overall_valid`, `axiom6_satisfied`, `kinetic_traps`, `full_cycle_candidacy`
    - * CLI: `syncon trajectory validate -steps <names> -reset <name>`
    - * CLI: `syncon trajectory criticality -steps <names> -varma-probe`
  - * **SYNTHONIC_ENSEMBLER** (`synthomnicon/ensembler.py`):`EnsembleCatalog` — N×N pairwise compatibility matrix for any number of synthons
    - * Three emergent properties: criticality (degeneracy_strength), G_→G_ amplification (Axiom 3), interface fidelity degradation
    - * Axiom propagation: Axioms 1, 2, 3, 5 evaluated at ensemble level
    - * System-level ξ_CP: uses highest-F component as reference; interface

overhead adds bits via Landauer identity
    - CLI: `syncon ensemble check -components <names>`
    - CLI: `syncon ensemble probe -criticality -components <names>`
    - CLI: `syncon ensemble thermo -components <names> -delta-g-assembly <float>`
  - **SYNTHONIC_RETRODESIGN** (`synthomnicon/retrodesign.py`):`RetrodesignE` — recursive axiom-pruned decomposition tree
    - Axiom pruning: Axiom 1 (Fidelity Floor), Axiom 2 (Propagation Barrier — only if sub-tuple claims G_), Axiom 4 (Grammar Mismatch), Axiom 6 (Grounding Fail)
    - Sibling compatibility: incompatible siblings pruned via `ConstraintEngine.check_pair_`
    - `DecompositionTree`: `valid_leaves`, `pruned_count`, `valid_synthon_set`
    - `SynthonNotation.from_string()` monkey-patched as static method alias for `parse_notation()`
    - CLI: `syncon retrodesign <name_or_notation> -max-depth 3 -prune-axioms 1,2,4,6`
  - **Demo script** (`examples/demo_protocols.py`):Exercises all four protocols with calibrated reference values
    - `carboxylic_acid_dimer` ΔG = -12.0 kJ/mol $\rightarrow$ ξ_CP $\approx$ 6.66 nats [HIGH]
    - `proline_aldol_cycle` three-step D_∞ validation with reset detection
    - Three-component ensemble: molecular + temporal + mechanical
    - Retrodesign with catalog lookup and notation string decomposition
  - `__init__.py` version bumped to `0.3.0`; all four modules exported in `__all__`
- **March 15, 2026 — v2.1.3: NLP Format Enforcement + CLI Fixes**:
  - **All LLM Prompts Now Follow NLP_FORMAT.md** — mandatory compliance enforced across entire codebase:`agents/axiom_guided_generator.py` — `_build_generation_prompt()` and `_get_system_prompt()` rewritten with XML tags (`<role>`, `<task>`, `<input>`, `<axioms>`, `<instructions>`, `<output_format>`, `<mechanistic_constraints>`), `**MUST**`/`**MUST NOT**` emphasis, declarative commands
    - `synthomnicon/llm_grounding.py` — `EXTRACTION_PROMPT` and `ADVERSARIAL_GROUNDIN` updated
    - `agents/autonomous_synthon_discovery_agent.py` — all 3 prompts updated
  - **Fix 1 CLI Flags Fully Implemented** (`synthomnicon/cli.py`):`-strict-grounding` — Blocks catalog registration if any primitives fail grounding
    - `-override-grounding` — Force registration despite failures (requires `-override-reason`)
    - `-override-reason TEXT` — Justification logged to audit trail
    - `-speculative` — Register in 'speculative' domain (quantum/hypothetical systems)
  - **Fix 6: Catalog Audit Command** — NEW `syncon audit` command:`-axiom 6` — Audit all D_∞ entries for closed-cycle grounding
    - `-axiom 7` — Audit all T_ entries for named closing bond
    - `-primitive / -value` — Filter by specific primitive type/value

    - * `-status` — Filter by grounding status (unverified, partial, override, etc.)
    - * `-auto-flag` — Set `flagged_for_review` on problematic entries
    - * `-dry-run` — Preview without making changes
  - * **Implementation Status** (from SYNTHONICON_FIXES.md):Fix 1: Registration block — Critical impact (CLI flags complete)
    - * Fix 2: Axiom 6 (D_∞) — High impact
    - * Fix 3: Axiom 7 (T_) — High impact
    - * Fix 4: Per-primitive confidence — Medium impact (`PrimitiveGrounding.confidence`, `ADVERSARIAL_GROUNDING_PROMPT`)
    - * Fix 5: Quantum quarantine — Medium impact
    - * Fix 6: Catalog audit — Medium impact (`syncon audit` command)
    - * Fix 7: Arbitrage mode — Low impact (future)
- **March 14, 2026 — SYNTHONICON_FIXES.md v2.1 Implementation** (Critical Fixes):
  - * **Fix 1: Registration Block on Grounding Warnings** (CRITICAL):Added `GroundingValidationError` exception class
    - * `SynthonCatalog.register()` now accepts grounding validation parameters
    - * Audit trail logs all grounding overrides with human-provided justifications
    - * Grounding metadata persisted in catalog JSON
  - * **Fix 2: Axiom 6 (Temporal Grounding)** (HIGH):D_∞ now requires closed cycle with reset mechanism
    - * Keyword indicators: `AXIOM_6_RESET_INDICATORS`, `AXIOM_6_PROCESS_INDICATORS`
    - * Validates justifications contain cycle description (reset + process)
    - * Canonical test cases: proline aldol cycle (PASS), extended allene wrong D_∞ (FAIL)
  - * **Fix 3: Axiom 7 (Cyclic Topology Grounding)** (HIGH):T_ now requires named closing bond/interaction
    - * Keyword indicators: `AXIOM_7_CLOSING_INDICATORS`, `AXIOM_7_INVALID_TOPO_KEYWORDS`
    - * Detects invalid justifications (linear, rod, chain, axial, etc.)
    - * Canonical test cases: carboxylic acid dimer (PASS), cumulene wrong T_ (FAIL)
  - * **Fix 4: Per-Primitive Confidence** (MEDIUM):`PrimitiveGrounding` dataclass extended: `confidence: float`, `is_grounded: bool`, `failure_reason`, `suggested_alternative`
    - * `ADVERSARIAL_GROUNDING_PROMPT` — challenges each primitive from first principles
    - * Per-primitive confidence auto-derived from `GroundingStatus` (GROUNDED=0.9, AMBIGUOUS=0.5, UNGROUNDED=0.1, INVALID=0.0)
  - * **Fix 5: Quantum Extension Quarantine** (MEDIUM):Added `domain` field to `CatalogEntry` dataclass
    - * Supports: molecular, supramolecular, temporal, hybrid, speculative, quantum
    - * Prevents semantic contamination of grounded catalog
  - * **New `AxiomResult` Dataclass**:Standardized axiom validation return type

    - Fields: axiom number, satisfied boolean, violations list, warnings list
  - **Files Modified**:`synthomnicon/registry.py` — Grounding validation, CatalogEntry, audit trail
    - `synthomnicon/constraints.py` — Axioms 6 & 7, AxiomResult, keyword constants
    - `synthomnicon/grounding.py` — Per-primitive confidence fields
    - `synthomnicon/llm_grounding.py` — ADVERSARIAL_GROUNDING_PROMPT (Fix 4)
- **March 13, 2026 — QUANTSYNTHONICON.md v2.0 Implementation** (Major Release):
  - · **Extended Primitive Framework** (7 → 9 primitives):Added **Kinetic Character (K)** — Separates thermodynamic/kinetic fidelityK_fast (<60 kJ/mol), K_mod (60-100), K_slow (>100), K_trap
    - · Extended **Interaction Grammar (Γ)** — Boolean algebra for partner selectionΓ_∧ (AND), Γ_∨ (OR), Γ_→ (SEQUENTIAL) with SPECIFIC/SELECTIVE/BROAD tiers
    - · Added **Criticality Phase (Φ)** — Identifies scale-free systems at G-D degeneracyΦ_sub (normal), Φ_c (critical), Φ_super (post-assembly)
    - · Refined **Polarity (P)** — Symmetric vs. pseudosymmetric self-complementarityP_pm (symmetric), P_pm_pseudo (pseudosymmetric)
  - **Composition Axioms Validation** — 5 axioms from QUANTSYNTHONICON.md Section IV:Axiom 1: Cyclic closure amplifies fidelity (T_–F rule)
    - Axiom 2: Local grammar blocks network propagation (G_–Γ barrier)
    - Axiom 3: Cooperative induction superlinearity (G_ → G_ transition)
    - Axiom 4: Sequential grammar requires temporal/catalytic dimension
    - Axiom 5: Criticality contracts primitive basis (G-D degeneracy)
  - **Axiom-Guided Generation** — NEW `AxiomGuidedGeneratorAgent`:Iterative refinement until all axioms satisfied
    - Reports violations and warnings
    - CLI: `syncon generate -axiom-guided`
  - **Criticality Detection** — NEW `synthomnicon/criticality.py`:Scale-free behavior detection
    - Correlation length estimation
    - CLI: `syncon criticality -all`
  - **Transformation #8 Probe** — NEW `synthomnicon/transformation8.py`:Rotaxane dethreading analysis
    - Steric cliff detection (R_ signature)
    - Reference values added to thermodynamics module
  - **CLI Enhancements**:`generate -axiom-guided` / `-a` flag for axiom validation
    - `criticality` command for scale-free analysis
    - Updated table display for 9 primitives
  - **Thermodynamics Updates**:`compute_eta_CP()` now uses effective fidelity (F_thermo × F_kinetic)

    - * `compute_kinetic_fidelity()` — NEW function
    - * Reference values for rotaxane_dethreading and critical_hbond_percolation
  - – * **Updated Documentation**:README.md — Updated for 9 primitives, axiom-guided generation
    - * USAGE.md — Comprehensive v2.0 usage guide
    - * commit.txt — Implementation summary
  - – **All integration tests passing** (6/6)
- **March 13, 2026** — Autonomous Synthon Discovery Agent:
  - – * **Autonomous Synthon Discovery Agent**: Self-directed agent that continuously proposes, validates, and registers novel synthons`syncon agents discover` — Run autonomous discovery campaigns
    - * Automatic duplicate detection and diversity enforcement
    - * Configurable limits (cycles, duration, confidence threshold)
    - * Focus areas for targeted exploration (e.g., "halogen bonding", "catalysis")
    - * Progress tracking with auto-save every N cycles
    - * **Result**: 1,287 synthons auto-discovered in single session
  - – * **Catalog Persistence**: Synthons now persist across sessionsAuto-saves to `~/.synthomnicon/catalog.json`
    - * Auto-loads on startup
    - * All registrations are permanent
  - – * **AI-Powered Synthon Generation**: `SynthonGeneratorAgent` uses LLM reasoning`generate` — AI-powered synthon generation from descriptions
    - * `generate-smiles` — AI-powered generation from SMILES strings
    - * `compare` — Side-by-side synthon comparison with thermodynamic analysis
    - * `catalog tree` — Tree-view catalog display
    - * `export` — Export catalog to JSON/CSV/YAML
  - – * **Provider Updates**:Fixed hardcoded model defaults (now uses config-driven defaults)
    - * Updated Google provider to `google-genai` package (deprecated `google-generativeai`)
    - * All providers now use provider-specific defaults from `provider_defaults.yaml`
  - – * **Documentation**: Added comprehensive guides:`AUTONOMOUS_DISCOVERY.md` — Autonomous agent documentation
    - * `AUTONOMOUS_DISCOVERY_SUMMARY.md` — Implementation summary
    - * `DIVERSITY_FIX.md` — Diversity enforcement documentation
    - * `CATALOG_PERSISTENCE_FIX.md` — Persistence implementation
    - * `GOOGLE_PROVIDER_UPDATE.md` — Google package migration
    - * `PLACEHOLDER_FIXES.md` — Hardcoded default fixes
  - – Successfully debugged `test_integration.py` — all tests passing (6/6)
  - – Added **adenine-thymine (A·T) base pair** reference to `QUANTSYNTHONICON.md`
- **March 14, 2026** — Axiom Validation for Autonomous Discovery (Critical Fix):
  - – **Problem**: Autonomous discovery agent was registering synthons with incorrect primitive assignments, causing false 100% matches between chemically distinct systems (e.g., nitroso cavitand incorrectly matched to transient anhydride

dissipative cycle)
  - **Root Cause**: LLM assigned primitives based on surface description similarity without axiom validation; physically impossible combinations were being registered
  - * **Solution**: Added axiom validation at registration time in `AutonomousSynthonDiscovery`
    1 enforcement: Cyclic self-complementary synthons (T_/P_±) cannot have low fidelity (F_ell)
    * **Axiom 4 enforcement**: Sequential grammar (Γ_→) requires temporal (D_∞) or catalytic (R_) dimension
    * Synthons violating hard axioms are **rejected** with detailed error messages
    * Other axiom violations are **flagged** in synthon metadata for review
  - **Bug Fix**: Fixed `AxiomValidator.validate_axiom4_sequential_grammar()` temporal dimension check
  - **Testing**: Added `test_axiom_validation.py` with comprehensive axiom validation tests (all passing)
  - **Documentation**: Added `AXIOM_VALIDATION_FIX.md` with full technical analysis
  - **Impact**: Prevents false positive matches from LLM misassignment; ensures all registered synthons satisfy composition axioms
- **March 14, 2026** — Aider Provider Integration (v2.1.2):
  - **New Provider**: Added Aider as first-class provider alongside Anthropic, DeepSeek, Qwen, Mistral, Google
  - * **AiderLLMProvider**: Direct LLM access via Aider's model configurationUses Aider's LiteLLM wrapper with lazy loading
    * Automatic caching and token usage tracking
    * No API key required (uses underlying LLM provider's keys)
  - * **AiderCodeAgent**: Git-native AI pair programming agentAutomatic commits with descriptive messages
    * Multi-file editing coordination
    * Test/lint integration
    * Repo map for context-aware changes
  - * **Task Routing**: Aider preferred for coding and refactoring tasks`coding`: aider → qwen → mistral → deepseek → anthropic
    * `refactor`: aider → anthropic → qwen → deepseek
  - * **Configuration**: Added to `provider_defaults.yaml` with model optionsClaude Sonnet 4.5 (default)
    * DeepSeek Chat
    * Ollama local models (privacy-sensitive)
  - **Documentation**: Added comprehensive `AIDER_PROVIDER.md` guide
  - **Testing**: Added tests for provider and agent creation
  - * **Files Modified**:`framework/aider_provider.py` — NEW: AiderLLMProvider
    * `agents/aider_code_agent.py` — NEW: AiderCodeAgent
    * `framework/enhanced_llm_provider.py` — Register Aider provider
    * `provider_defaults.yaml` — Aider configuration

* `.env.example` — Aider environment variables
* `test_integration.py` — Aider tests

0.17 License

This project is part of the SynthOmnicon research framework. See the main repository for licensing details.